certora

Security Audit Report

Kleros V2

December 2025

Prepared for Kleros

Table of contents

Audit Report 3
- **Project Summary** 3
- Project Scope 3
- Protocol Overview 5
- Findings Summary 7
- Severity Matrix 7
- High Severity Issues 8
- H-01. Anyone can inflate totalStaked without depositing tokens 8
- H-02 Jurors can dominate dispute draws without long-term commitment 11
- **Medium Severity Issues** 13
- M-01. Malicious arbitrated contract can DoS executeRuling and prevent fee withdrawals 13
- M-02. Inflated totalStaked accounting 14
- M-03. Unchecked low-level safeTransfer call can lead to loss of tokens 15
- M-04. Change of court parameters can disrupt ongoing disputes 17
- M-05. Drawing probability ignores locked stake causing unfair juror allocation 18
- M-06. Locked stake isn't deducted from sortition weight 19
- M-07. Tied rounds make everyone coherent in >=3 choice disputes 20
- M-08. Appeal crowdfunding includes all options, regardless of their funding progress 22
- **Low Severity Issues** 24
- L-01. Missing whenNotPaused modifier in core functions 24
- L-02. Missing staleness check to calculate dispute fees 25
- L-03. Order of execution results in different outcomes 26
- L-04. Stake reward bypasses maxStakePerJuror limit in setStakeReward() 27
- L-05. Temporary fund unavailability for delayed unstakers 28
- **Informational Issues** 30
- I-01. NFT whitelist can be bypassed due to lack of continuous ownership validation 30
- I-02. Non-PoH jurors can DoS dispute drawing in Sybil-resistant courts 30
- I-03. Partial coherence formulas make certain disputes a lose-lose 31

Disclaimer 33

About Certora 33

Audit Report

Project Summary

This document describes the manual code review findings of **Kleros V2**. The work was undertaken from **November 12, 2025,** to **December 19, 2025**.

During this time, Certora's security researchers performed a manual audit of the contracts in scope and discovered bugs in the codebase, which are summarized in the subsequent section.

Project Scope

Project Name	Repository (link)	Commit Hash	Final Commit Hash	Platform
Kleros V2	/contracts/	592243f	2ae2b6a	EVM

The following contract list is included in our audit scope:

```
src/
├── arbitration
│   ├── arbitrables
│   │   ├── ArbitrableExample.sol ❌
│   │   └── DisputeResolver.sol
│   ├── devtools ❌
│   ├── dispute-kits
│   │   ├── DisputeKitClassic.sol
│   │   ├── DisputeKitClassicBase.sol
│   │   ├── DisputeKitGated.sol
│   │   ├── DisputeKitGatedArgentinaConsumerProtection.sol
```

```
│   │   ├── DisputeKitGatedShutter.sol
│   │   ├── DisputeKitShutter.sol
│   │   └── DisputeKitSybilResistant.sol
│   ├── evidence
│   │   ├── EvidenceModule.sol
│   │   └── ModeratedEvidenceModule.sol ❌
│   ├── interfaces
│   │   ├── IArbitrableV2.sol
│   │   ├── IArbitratorV2.sol
│   │   ├── IDisputeKit.sol
│   │   ├── IDisputeTemplateRegistry.sol
│   │   ├── IEvidence.sol
│   │   └── ISortitionModule.sol
│   ├── DisputeTemplateRegistry.sol
│   ├── KlerosCore.sol
│   ├── PolicyRegistry.sol
│   ├── SortitionModule.sol
│   ├── university ❌
│   └── view
│       └── KlerosCoreSnapshotProxy.sol ❌
├── gateway ❌
├── kleros-v1 ❌
├── libraries
│   ├── Constants.sol
│   ├── SafeERC20.sol
│   ├── SafeSend.sol
│   └── SortitionTrees.sol
├── proxy
│   ├── Initializable.sol
│   ├── KlerosProxies.sol ❌
│   ├── mock ❌
│   ├── UUPSProxiable.sol
│   └── UUPSProxy.sol
```

```
├── rng
│   ├── BlockHashRNG.sol ❌
│   ├── ChainlinkConsumerBaseV2Plus.sol
│   ├── ChainlinkRNG.sol
│   ├── IncrementalNG.sol ❌
│   ├── IRandomizer.sol ❌
│   ├── IRNG.sol
│   ├── mock ❌
│   ├── RandomizerRNG.sol ❌
│   └── RNGWithFallback.sol
├── test ❌
├── token ❌
└── utils
    └── TransactionBatcher.sol ❌
```

Protocol Overview

Kleros is an on-chain dispute resolution and arbitration protocol on the EVM. It enables users to become jurors in resolving debates on-chain by staking the system-native token, PNK, present evidence and vote in favour of a side. Its value proposition includes:

- Staking PNK in courts so they can be randomly selected to participate in disputes
- Users who voted coherently are rewarded penalties gathered from incoherent jurors.
- Facilitate appeals in case of a tie or if the outcome is considered unfair

Key Features:

- Staking PNK in numerous courts simultaneously
- Arbitrary juror selection based on Chainlink VRF
- Appeal in case a round is considered unfairly judged
- Court and dispute kit jumps, changing parameters such as juror count, coherence formulas, hidden vs public voting, increasing fees, etc.

Architecture:

- Kleros Core: User entry point, handles staking/unstaking logic, creation of disputes and courts, drawing and execution
- Sortition Module: Sortition tree based contract, keeps track of user stakes across courts, user chances of being drawn are proportional to their stake size
- DisputeKitClassicBase: Abstract contract serving as foundation for every dispute kit. Responsible for tracking dispute progress, casting votes, funding appeals and determining degree of coherence for jurors

The protocol uses a round-based system where each dispute has multiple rounds until a consensus is reached. After an ultimate winner is decided, the execution period determines the degree of coherence in each round. A juror is considered coherent in a round if they voted for the ultimate winner, regardless if they were a minority initially e.g Dispute has 3 choices - [A, B, C], we vote for choice A in the first round which is the least popular one, next appeal choice A is most voted for and becomes the winner - we are coherent in the first round too since we chose the ultimate winner from the start. Incoherent users are penalized for voting incorrectly and penalties are distributed evenly among coherent jurors.

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	0	0	0
High	2	2	1
Medium	8	6	5
Low	5	3	3
Informational	3	2	0
Total	–	–	–

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

High Severity Issues

H-01. Anyone can inflate totalStaked without depositing tokens

Severity: **High**	Impact: **High**	Likelihood: **High**
Files: KlerosCore.sol#L1341-L1364	Status: Fixed	

Description

When processing delayed stakes, `totalStaked` can be permanently inflated without actual token deposits due to state changes occurring before token transfer validation. The issue occurs because `executeDelayedStakes` processes delayed stakes without reverting:

```javascript
        for (uint256 i = delayedStakeReadIndex; i < newDelayedStakeReadIndex; i++) {
            DelayedStake storage delayedStake = delayedStakes[i];
       @>    if (!core.setStakeBySortitionModule(delayedStake.account, delayedStake.courtID,
delayedStake.stake)) {
                emit StakeDelayedExecutionFailed(delayedStake.account, delayedStake.courtID,
delayedStake.stake);
            }
            delete delayedStakes[i];
        }
        delayedStakeReadIndex = newDelayedStakeReadIndex;
    }
```

But in `KlerosCore._setStake()`, the token transfer happens after `validateStake()` has already modified `totalStaked`:

```javascript
(uint256 pnkDeposit, uint256 pnkWithdrawal, StakingResult stakingResult) =
sortitionModule.validateStake( // @audit - validate stake modifies totalStaked
            _account,
            _courtID,
            _newStake,
            _noDelay
        );
        if (stakingResult != StakingResult.Successful && stakingResult !=
StakingResult.Delayed) {
            _stakingFailed(_onError, stakingResult);
            return false;
        } else if (stakingResult == StakingResult.Delayed) {
            return true;
        }
// @audit - token transfer fails due to lack of funds
        if (pnkDeposit > 0) {
            if (!pinakion.safeTransferFrom(_account, address(this), pnkDeposit)) {
                _stakingFailed(_onError, StakingResult.StakingTransferFailed);
                return false;
            }
        }
```

This allows a malicious juror to:

1. Create delayed stakes without sufficient PNK balance
2. When executed, `totalStaked` increases during `validateStake()`
3. Token transfer fails, function returns `false`
4. `totalStaked` remains permanently inflated

5. A permanent DoS is possible if `maxTotalStaked` is `type(uint256).max` as is set in the tests.

Customer's response: Confirmed and fixed here

Fix Review: Fix looks good, totalStaked is now incremented after checking for successful token transfer.

H-02 Jurors can dominate dispute draws without long-term commitment

Severity: **High**	Impact: **Medium**	Likelihood: **High**
Files: [KlerosCore.sol#L747-L776](#)	Status: Confirmed, fix in future implementation	

Description: Juror selection is purely stake-weighted and does not account for how long PNK has been staked. If a dispute is created near the end of the staking phase, a juror can stake a large amount of PNK shortly after and become eligible for that dispute, dominating the draw. With non-Sybil dispute kits, this juror can be selected multiple times. This enables short-term actors to enter only for opportunistic disputes, capture ETH and PNK rewards, and exit once the execution period is over, while long-term stakers receive no proportional advantage for maintaining continuous stake.

Recommendations: Implement a cooldown period requiring newly staked PNK to wait before becoming eligible for dispute draws.

Customer's response: Yes confirmed for both Kleros v1 and v2, although in the described scenario it is more about dominating "without any staking history" rather than "without long-term commitment". We have decided not to make distinctions based on longer-term staking history, such as giving extra weight to a juror who has been staked for years than one who has only been staked for months, as this represents tradeoffs in the security model, making it harder for newcomers to influence specific disputes but creating the potential for new types of long-range attacks.

Fix review: Acknowledged, the team noted future improvements to be introduced in the shape of an exit queue.

Medium Severity Issues

M-01. Malicious arbitrated contract can DoS executeRuling and prevent fee withdrawals		
Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: KlerosCore.sol#L1071	Status: Fixed	

Description: When `KlerosCore.arbitrableWhitelistEnabled` is false, `KlerosCore` accepts any contract as arbitrated without validation. A malicious arbitrated contract can intentionally revert in its `rule()` callback, preventing `KlerosCore.executeRuling()` from completing. This blocks the dispute from being marked as ruled, which in turn prevents legitimate jurors and appeal funders from withdrawing their fees and rewards through `withdrawFeesAndRewards()`.

Recommendations: Wrap the external call to `dispute.arbitrated.rule()` in a try-catch block with a gas limit to prevent malicious contracts from blocking execution. This ensures the dispute is always marked as ruled regardless of the arbitrated contract's behavior.

Customer's response: Confirmed and fixed here

Fix review: Fix looks good, try-catch was implemented to ensure non-failure.

M-02. Inflated totalStaked accounting

Severity: **Medium**	Impact: **Low**	Likelihood: **High**
Files: (1), (2), (3)	Status: Fixed	

Description:

Currently `totalStaked` changes take place in `validateStake` which is performed on most, but not all PNK movements within the codebase.

Examples of such functions are `withdrawLeftoverPNK`, sending rewards directly during `_executeRewards` and whenever leftover PNK is sent to the owner when coherence is (0, 1). Neither of these flows interact with `validateStake` or influence `totalStaked`, however they perform changes to `juror.stakedPnk` and PNK balance of the contract. Ideally, the sum of all juror's staked PNK should amount to `totalStaked`, however this won't be the case here.

Customer's response: Confirmed and fixed here

Fix Review: Fix looks good, every PNK movement is now properly reflected in accounting

M-03. Unchecked low-level safeTransfer call can lead to loss of tokens

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: SafeERC20	Status: Partially fixed	

Description: The protocol currently uses a `safeTransfer` function that performs a low-level call to the token address and returns a boolean of its success. However, this success is not checked on every use of `safeTransfer` across the protocol. For instance `withdrawLeftoverPnk`, `transferBySortitionModule`, `_transferFeeToken`.

```javascript
        function safeTransfer(IERC20 _token, address _to, uint256 _value) internal returns
(bool) {
         (bool success, bytes memory data) =
address(_token).call(abi.encodeCall(IERC20.transfer, (_to, _value)));
         return (success && (data.length == 0 || abi.decode(data, (bool))));
    }
```

A token transfer can fail for multiple reasons (e.g., pausable ERC20s, insufficient balance, non-standard ERC20 behavior). If the success flag is ignored, these failures become silent and can result in a complete loss of user funds.

Recommendations: Ensure the return value is checked. If a transfer fails, consider recording the failed amount in a mapping and exposing a dedicated function that allows users to recover their tokens later.

Customer's response: Confirmed and partially fixed here. Failure on exceptional token circumstances have not been patched as it would add too much complexity. If such an event takes place, funds can be saved by executing governance proposals to rescue them.

Fix review: Fix looks good, the unfixed parts make sense and having a backdoor in case of an emergency is sufficient given the acceptable risk.

M-04. Change of court parameters can disrupt ongoing disputes

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: KlerosCore.sol#L509-L542	Status: Fixed	

Description: The `KlerosCore.changeCourtParameters` function allows modifying court parameters without checking for active disputes. When parameters are changed while disputes are ongoing, those disputes experience rule changes mid-process: vote mechanisms can switch modes, period deadlines shift, and appeal behavior changes, violating the principle that dispute rules should remain constant from initiation to resolution.

Recommendations:

Add a validation check in `changeCourtParameters` that reverts if the court has any active disputes.

Customer's response: Confirmed and fixed here and here (combined diff)

Fix review: Fix looks good, relevant parameters are stored in-round.

M-05. Drawing probability ignores locked stake causing unfair juror allocation

Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: [SortitionModule.sol#L472-L481](#)	Status: Acknowledged	

Description: The drawing mechanism in `SortitionTrees.draw()` assigns selection probability based on a juror's total staked PNK without considering how much is already locked in active disputes. A juror with 10,000 PNK staked across multiple ongoing cases (9,500 locked, 500 available) has the same draw weight as a juror with 10,000 PNK fully unlocked. This creates systematic overallocation of busy jurors while underutilizing available ones.

Recommendations: Adjust sortition tree weights to reflect unlocked stake (`stakedPnk - lockedPnk`) rather than total stake. Update `lockStake()` and `unlockStake()` to recalculate and propagate weight changes through the tree, ensuring draw probability correlates with actual juror availability.

Customer's response: This choice has been made by design to prevent potentially malicious parties from being able to reduce the pool of available jurors by creating irrelevant disputes. If jurors' draw probabilities were reduced to reflect locked amounts, an attacker could potentially attempt DoS attacks where not enough jurors are available, or even stake themselves at a strategic moment in order to gain a disproportionate probability of being drawn.

Certora: Acknowledged.

M-06. Locked stake isn't deducted from sortition weight

Severity: **Medium**	Impact: **Medium**	Likelihood: **Medium**
Files: KlerosCore.sol#L757-L766	Status: Acknowledged	

Description: Whenever jurors are drawn, `KlerosCore.draw()` calls: `sortitionModule.lockStake(drawnAddress, round.pnkAtStakePerJuror);` but `lockStake()` only executes: `jurors[_account].lockedPnk += _relativeAmount;` and leaves the juror's full stake amount inside the sortition sum tree unchanged. Because the locked portion is never removed from the weight that future draws use, each draw keeps treating the same tokens as available collateral, allowing one juror to occupy multiple vote slots across disputes while penalties cap at their total stake, but rewards accumulate for each position.

Recommendations: Deduct the locked amount from the sortition-tree weight when a juror is drawn so the same PNK can't back multiple votes. Additionally, enforce `lockedPnk + amount <= stakedPnk.`

Customer's response: Over-locking is allowed by design. This is a trade-off which balances the interest of the arbitrated protocol and the jurors. Over-locking is not ideal, but it prevents other attacks which would succeed if it were changed.

Fix review: Acknowledged.

M-07. Tied rounds make everyone coherent in >=3 choice disputes

Severity: **Medium**	Impact: **High**	Likelihood: **Low**
Files: DisputeKitClassicBase	Status: Fixed	

Description:

A round is considered tied whenever some choice's count matches the current `winningChoice` count. Once a round is considered tied and executed, it treats all voted jurors as coherent

```javascript
        } else if (tied) {

            return currentRound.totalVoted;

}
```

This is counter-intuitive in disputes with more than 2 choices. If a dispute has vote options [A, B, C, D, E] with votes spread as:

- 50-49-10-10-10 - Choice A is `winningChoice` everyone else is incoherent and subject to penalties

- 50-50-10-10-10 - A juror votes for choice B, state becomes `tied = true`, suddenly CDE are considered coherent as well

This creates weird incentives to try and enforce ties instead of supporting what a juror believes in, in order to avoid penalties e.g it's more profitable for choice C supporters to vote for B just to avoid penalties.

Recommendation:

Revisit the logic, consider counting only the actual tied choices as coherent rather than every juror who voted

Customer's response: Confirmed and fixed here

Fix review: Fix looks good, winning choices are now stored separately in a dynamic array. If either of them gets overthrown, the array is deleted and the winner is stored.

M-08. Appeal crowdfunding includes all options, regardless of their funding progress

Severity: **Medium**	Impact: **Low**	Likelihood: **High**
Files: [DisputeKitClassicBase]()	Status: Acknowledged	

Description:

In order for an appeal to start, at least 2 choices must be funded in a crowdfunding round. Any partially filled choices are refunded. However the current logic disregards which choice was funded and which was not and starts a new dispute round with all of the original ones

```JavaScript
        if (round.fundedChoices.length > 1) { // triggers as soon as 2 choices are funded
            round.feeRewards = round.feeRewards - appealCost;

            (, , , , bool isDisputeKitJumping) =
core.getCourtAndDisputeKitJumps(_coreDisputeID);
            if (isDisputeKitJumping) {

                coreDisputeIDToActive[_coreDisputeID].currentRound = false;
            } else {
                dispute.coreRoundIDToLocal[coreRoundID + 1] = dispute.rounds.length;
                Round storage newRound = dispute.rounds.push();
                newRound.tied = true;
            }
            core.appeal{value: appealCost}(_coreDisputeID, dispute.numberOfChoices,
dispute.extraData); // however, it uses original dispute.numberOfChoices
        }
```

From the snippet above we see that the `appeal` functions are invoked with the original `dispute.numberOfChoices`, disregarding whether they were funded or not. This creates no incentive to try and fund a less popular choice if another 2 would be appealed either way, creating a free-riding scenario.

Recommendation

Include only successfully funded choices in the appeal.

Customer's response: Not eliminating alternative options that are not funded is a design choice. Both alternatives are not ideal and there might be dispute kits implemented in the future that implement both cases.

Fix review: Acknowledged

Low Severity Issues

L-01. Missing whenNotPaused modifier in core functions		
Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: KlerosCore.sol#L276-L279	Status: Fixed	

Description: Several functions in `KlerosCore` are missing the `whenNotPaused` modifier, including `createDispute`, `passPeriod`, `draw`, `appeal`, `setStakeBySortitionModule`, and `transferBySortitionModule`. As a result, when the protocol is paused, state-changing operations can still execute. For example, `setStake` is paused but delayed stakes can still bypass it via `setStakeBySortitionModule`, disputes can still be created, periods advanced, appeals made, and jurors drawn. This breaks the expected guarantees of the pause mechanism and can lead to inconsistent protocol state during emergencies.

Recommendations: Apply `whenNotPaused` to all state-changing functions in `KlerosCore`, including those reachable indirectly (e.g. via `SortitionModule`).

Customer's response: Confirmed and fixed here and here. The proposed fix was not fully correct, the rationale for the existing pausing mechanism was to preserve the liveness of the dispute resolution process but only pause token/value flow in and out, the threat model being to guard against loss of funds.

Fix review: Fix looks good, modifiers are added to suit the needs of the protocol. An additional modifier `whenArbitrationNotPaused` was added for better flexibility.

L-02. Missing staleness check to calculate dispute fees

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: KlerosCore.sol#L1230-L1232	Status: Fixed	

Description: The `Kleros.convertEthToTokenAmount` function converts ETH-denominated arbitration fees to token amounts using manually-set exchange rates. These rates are updated only by the contract owner through `changeCurrencyRates` and have no timestamp or staleness validation. When users create disputes paying with alternative tokens, stale rates cause incorrect fee calculations, likely resulting in disputes being created with significantly more or fewer jurors than economically intended.

Recommendations: Consider implementing an oracle.

Customer's response: Price oracle integration preparatory work started, fixes here and here (combined diff)

Fix review: Fix looks good, the team has implemented a new RateConverter contract to handle pricing without touching KlerosCore. Oracle integration is considered in future updates.

L-03. Order of execution results in different outcomes

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
Files: KlerosCore	Status: Acknowledged	

Description: Current design introduces overlocking and users being subject to more penalties than what they have at stake. Given that penalties are capped to existing funds and rewards are always executed after penalties, this creates certain cases where if 2 disputes [A, B] are executed in different order, it would result in different stake/rewards/penalties sizings e.g A user who has `stakedPnk = 100` and upcoming 200 penalties and 200 rewards should continue being at 100 staked tokens given they are delta neutral on their earnings-losses. However, if penalties are executed first, they would be capped at their current stake and afterwards they would be awarded 200 tokens, resulting in either `stakedPnk = 200` or them being directly sent to their wallet.

Recommendations: Revise the design of having penalties capped before sending out rewards.

Customer's response: Cannot improve without a major rewrite and it's not clear what the solution would look like. Some outcomes are better for some jurors, but there's no outcome that's better for every jurors in general. There's no right or wrong order of execution. If we force an order of execution then it's more deterministic for sure, but it's not necessarily fairer. And that might be a DoS vector if a specific dispute fails to `execute()` for any reason.

Fix review: Acknowledged, impact is somewhat negligible and a major rewrite risks introducing new attack vectors.

L-04. Stake reward bypasses maxStakePerJuror limit in setStakeReward()

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
SortitionModule.sol#L374-L375	Status: Fixed	

Description: In `SortitionModule.setStakeReward()`, the new stake is calculated as `currentStake + _reward` and passed directly to `_setStake()` without checking against `maxStakePerJuror`. This bypasses the validation that exists in `validateStake()`, allowing jurors who have already reached the maximum stake to exceed it through coherent juror rewards.

Recommendations: Return false if `newStake > maxStakePerJuror`, this makes `KlerosCore` to transfer reward tokens directly to the juror instead of increasing their stake.

Customer's response: Confirmed and fixed here

Fix review: Fix looks good, accounting is now properly updated and limits are adequately enforced.

L-05. Temporary fund unavailability for delayed unstakers

Severity: **Low**	Impact: **Medium**	Likelihood: **Low**
KlerosCore	Status: Acknowledged	

Description:

Whenever users request an unstake, the amount of PNK unstaked is capped to their available `staked - locked` in `validateStake`

Essentially if a user has `stakedPnk = 200` and `lockedPnk = 150` he would withdraw at most 50 PNK. The issue arises when supplying `_newStake = 0` during unstaking, which essentially means the user wants to withdraw all of his available tokens. If the user has locked tokens, his withdrawal would be capped to the available ones, however their stake in the sortition tree will be entirely wiped.

There is the function `withdrawLeftoverPNK` which is designed for jurors to pull tokens in such scenarios. Whether a juror can pull their tokens is determined by `getJurorLeftoverPNK`

```JavaScript
        function getJurorLeftoverPNK(address _juror) public view override returns (uint256) {
        Juror storage juror = jurors[_juror];
        if (juror.courtIDs.length == 0 && juror.lockedPnk == 0) {
            return juror.stakedPnk;
        } else {
            return 0;
        }
    }
```

The issue here is that the function would return a jurors `stakedPnk` only if they are out of all courts and have no locked PNK (meaning they are not participating in any disputes right now). This creates a problem for users who participate in more than one court, who will be forced to exit all courts if they wish to pull their funds from the specific one they initiated a full unstake from. Likelihood and impacts are low since it requires a user mistake and there is no material loss.

Impact

Dysfunctional happy path

Mitigation

If the withdrawal is capped to the available funds, adjust the `_newStake` amount too.

Customer's response: For a juror whose stake is partially locked in a dispute and who attempts to unstake everything: his whole stake will be immediately removed from the Sortition tree (such that he won't be drawn anymore) but his locked stake remains on the contract until the dispute resolves. At this point this "leftover" can be withdrawn with the withdrawLeftoverPNK() as soon as the juror is not staked in any court anymore. We are planning to automate this so the jurors don't need to be exposed to this upkeep task.

Fix review: Acknowledged

Informational Issues

I-01. NFT whitelist can be bypassed due to lack of continuous ownership validation

Description:

The `KlerosCore.setStake` function validates NFT ownership via `jurorNft.balanceOf(msg.sender) == 0` only at the moment of staking, without recording the specific NFT ID or re-validating ownership thereafter. A juror can stake, transfer their NFT to another address, which can then stake using the same NFT. This enables a single NFT to authorize multiple active stakes, breaking the intended 1-to-1 relationship between NFT holders and staked jurors.

Recommendations: Restrict the NFT from being transferred.

Customer's response: The actual NFT issued by the Galxe platform is non-transferable and outside the scope of the audit. Therefore it is safe, nothing to fix.

Fix review: Acknowledged

I-02. Non-PoH jurors can DoS dispute drawing in Sybil-resistant courts

Description:

In `DisputeKitSybilResistant`, jurors not registered in Proof of Humanity return address(0) from `_postDrawCheck()`, wasting draw iterations. A juror holding a majority stake (e.g., 98% of PNK) in a Sybil-resistant court without PoH registration causes most draw attempts to fail. Once `maxDrawingTime` passes, anyone can call `SortitionModule.passPhase()` to transition back to staking, forcing the dispute to wait through another full phase cycle (staking → generating → drawing) before drawing can resume.

Recommendations: Exclude non-PoH registered jurors from the sortition tree in Sybil-resistant courts, or implement a fallback mechanism that extends the drawing phase when insufficient valid jurors are drawn.

Customer's response: Confirmed and fixed here. The new implementation now forbids ineligible users to stake at all without the PoH check.

Fix review: Fix looks good, issue is patched on stake level rather than complicating the draw level by distinguishing between eligible and ineligible stakers. Now all jurors would be checked for PoH before they even become jurors.

I-03. Partial coherence formulas make certain disputes a lose-lose

Description: Currently partial coherence cases could result in jurors netting losses regardless of their vote choice. This happens due to several factors:

Partially coherent jurors are subject to `minStake * (1-coherence) penalty`
Their rewards are a mix of all penalties gathered from partially coherent and partially incoherent jurors, adjusted for their own coherence
There is no guarantee they would end up with more than what they were penalized initially, however all leftover tokens are passed to the owner.

Example:
3 users, pnkAtStake per person is 200
Votes are 2-1, with coherence 55-45
pnkPenalties = 200 * 0.45 + 200 * 0.45 + 200 * 0.55 = 90 + 90 + 110 = 290
reward per coherent user = 290 / 2 * 0.55 = 79.75

Partially coherent users were penalized 90 tokens and received back ~80. Every juror ends up at a loss while the protocol owner receives 290 - (2 * 79.75) = 159.5 tokens. The issue stems from the fact that partially coherent jurors are charged `minStake * (1-coherence)` as penalty, from which they receive `penalty * coherence` back, also expressed as `minStake * (1-coherence) * coherence`. The rest of the rewards come from incoherent users which do not guarantee to be enough to make coherent ones at least breakeven.

Recommendations: Ensure logic for new dispute kits does not depend on partial coherence.

Certora: After discussion with the Kleros team, we came to an agreement that all issues related to partial coherence are graded as informational due to the in-scope kits supporting coherence being either 1 or 0. It is the responsibility of future dispute kit implementations to handle such cases once partial coherence goes live.

Customer's response: No impact on kits in scope. It is the responsibility of the future dispute kit implementations to produce fair coherent values, it is not the concern of KlerosCore.

Fix review: Acknowledged.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline. It is helpful for smart contract developers and security researchers during auditing and bug bounties.

Certora also provides services such as auditing, formal verification projects, and incident response.